Review prep — context engineering, caching and compaction

Everything here is **measured** from `inv_2a6e5e42bebc480a94fa5ecd5e0457cb`, the run that found the `startTime` fault on 2026-10-28, unless marked otherwise. Numbers come from the `usage` column of that investigation, written by `engine/llm/metering.py`.

---

1. The run, in numbers

	Measured
Model calls	73 (6 failed and retried)
Input tokens	292,904
Output tokens	20,354
Total	**313,258**
Served from provider cache	**0**
Not re-sent, thanks to compaction	6,600
Desks that ran	7 + supervisor + RCA

Per desk:

Desk	**Calls**	**Input**	**Output**	**Saved by compaction**
observability	22	80,890	4,248	0
frontend_hosting	14	80,111	2,165	77
knowledge	8	36,894	2,383	6,523
backend_services	11	26,957	2,664	0
code	4	27,633	1,167	0

Desk	Calls	Input	Output	Saved by compaction
database	10	26,041	1,470	0
supervisor	2	4,098	2,514	0
rca	**2**	**10,280**	3,743	0

Learn two of these: **313k total**, and **the RCA used 10,280 input tokens — 3.5% of the run.** The second number is the one that demonstrates the architecture.

---

2. "What context engineering did you use?"

Four mechanisms. Name them in this order; the first is the big one.

a. Per-desk context isolation — the primary lever

Each desk has **its own message list**. Nothing from one desk's window ever enters another's. Only cited one-line findings cross between desks, through `ask_agent` (`engine/questions.py`).

The proof is in the table: the RCA agent, which writes the actual answer, spent **10,280 input tokens against the run's 292,904**. It never sees raw tool output at all — it reads evidence lines only (`engine/rca.py`). A single-agent design would have carried all seven desks' tool output into the final reasoning step.

> *Say it like this:* "Isolation is what keeps the window small. The agent that produces the answer reads 3.5% of the run's tokens, because it reads evidence, not logs."

b. Per-tool output caps

Every tool declares its own ceiling in the catalog (`ToolMapping.max_output_chars`), enforced in `capabilities/runner.py:67` before output ever reaches a model.

- A build log may spend 16,000 characters
- A deployment list: 8,000
- A branch or zone list: 2,000
- Default: `Budgets.tool_output_chars` = 8,000

Truncation is **declared, not silent**: the result is marked `truncated: true`, the agent sees `… [truncated N characters]`, and the event stream records it. The desk can call the tool again more narrowly.

c. Cache-stable system prompts

The system prompt is **byte-identical across every turn of a desk** (`engine/prompts.py:19`). Anything per-call — the clock, the task — goes in the first *user* message (`prompts.now_line`), never the system prompt. This exists so a provider can cache the prefix.

This is a design decision with a cost: it is slightly less convenient to write prompts this way. We accepted that because a moving system prompt makes caching impossible.

d. Deliberate scope limits

- `Budgets.specialist_tool_calls` = 8 per desk
- `empty_results_before_stop` = 2 — a desk that gets nothing useful twice stops searching that source (`engine/specialist.py`, `_exhausted_search`)
- `engine/triage.py` answers platform questions like "what is connected?" from the database with **zero model calls**

---

3. "What compaction did you use, and on what?"

Rolling compaction of tool output inside a desk's window — `engine/specialist.py:166`, `_view()`.

How it works:

1. The newest `keep_full_tool_outputs` (default **3**) tool results stay complete.
2. Every older tool result collapses to its first `collapsed_output_chars` (default **500**).
3. **The `call_id` survives the collapse.** This matters: a finding must cite `tool_call_ids`, so a desk can still cite evidence whose body has been compacted.
4. The full result is never lost — it stays in the `ToolCallRecord`, so the report and the evidence wall still show it.

It is lossy on purpose. A desk that needs a dropped detail calls the tool again: one tool call, versus re-sending that output on every remaining turn.

What it is not: no model is used to summarise. Compaction is deterministic string truncation, so it costs zero tokens to perform. That is a deliberate difference from summarisation-based compaction, which spends inference to save inference.

The honest number

Compaction saved **6,600 tokens of 292,904 — about 2.2%**, almost all of it in the knowledge desk (6,523).

Be ready to explain why it is small, because someone will ask:

> Compaction only triggers on a desk's **fourth** tool result. Most desks in this run made fewer than four calls, or made calls returning small payloads. The knowledge desk searched a vault and got large text back, so it is where compaction paid. The mechanism is sized for long investigations; this one was short.

That is a better answer than claiming a large saving.

---

4. "What caching did you use?"

Implemented: provider prompt caching. `engine/llm/openrouter.py:_system_message` marks the system prompt as the cache breakpoint with `cache_control: {"type": "ephemeral"}` when `FORGEOPS_PROMPT_CACHE=true`.

Measured in this run: `cached_tokens = 0`.

Do not hide this. Say it directly:

> "Caching is implemented and switched off in this run, because every role is on a `:free` OpenRouter model and free-tier models do not support prompt caching. We measure it rather than assume it — the meter reports cached tokens per desk, and it reported zero. On a paid model the stable system prefix is what the cache would hit."

If asked *how much would it save?* — say you have not measured it, and that published figures for cached input are around a 50–90% discount depending on provider, so the honest answer is "we expect a large reduction in input cost but we have not run the experiment." **Do not state a number as if it were yours.**

There is a second caching fact worth knowing: a research result (TokenPilot, arXiv 2606.17016) shows that **aggressive compaction can invalidate a cached prefix**, because rewriting history changes the tokens. ForgeOps avoids that collision by compacting only *tool output in the user/tool messages* and never touching the system prefix. That is a real design point, not a retrofit.

---

5. "Why 313k tokens? Claude Code does bigger tasks for less."

The strongest answer is structural, not defensive:

1. **Seven desks, seven windows.** A single agent keeps one history. ForgeOps paid for a system prompt plus tool schemas *per desk, per turn*. Published practitioner estimates

put multi-agent systems at roughly 15× a single agent for this reason.
2. **No cache in this run** (see §4). Comparisons against a cached single agent are not like for like.
3. **Per-call size is already lean:** 292,904 ÷ 73 = **4,012 input tokens per call**. The cost is in the *number* of calls, not their size — which is why the next optimisation targets call count, not prompt length.

Then pivot to what the architecture buys: **parallel independent investigation with cited evidence, and four desks that corroborate each other.** The fault was found because Code, Database, Knowledge and Frontend each held one piece.

---

6. Questions you should expect, with short answers

"Is compaction summarisation?" No. Deterministic truncation to the first 500 characters, keeping the `call_id`. No model call, so it costs nothing to perform.

"How do you not lose evidence when you compact?" The `call_id` survives, the full output stays in the tool call record, and `SpecialistLoop._accept` rejects any finding that cites an id it cannot resolve. Compaction can shrink what the model *sees*; it cannot break the citation chain.

"How do you know any of these numbers?" `MeteredLLM` wraps every model call and attributes it to a desk; totals are written to the investigation row and streamed as events. Nothing here is estimated.

"What is the next optimisation?" Fewer calls, not smaller prompts. Specifically: deterministic evidence pre-fetch — run the obvious reads with no model turn, then let each desk reason once over a prepared pack. The literature on LLM-based RCA supports this; one graph-based system invokes the model for only 14–20% of candidates.

"What did you not check?" Cloudflare Workers Observability reported zero errors for this fault, correctly: the failure happens in the browser talking directly to Supabase, so Cloudflare never sees it. The run says so rather than inventing a signal. That is the "say what you could not check" rule.

---

7. One-sentence summaries to memorise

- **Context engineering:** per-desk isolation, per-tool output caps, cache-stable system prompts, and hard budgets — the RCA reads 3.5% of the run's tokens because it reads evidence, not logs.

- **Compaction:** deterministic, zero-cost, keeps the newest three tool results whole and collapses older ones to 500 characters while preserving `call_id`; saved 6,600 tokens here.
- **Caching:** implemented as a stable system prefix with `cache_control`; measured zero this run because free models do not support it.